

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Лабораторная работа №2
«Реализация микросервисной архитектуры»

Выполнила:

Олейник Полина

Группа 2.1

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Реализовать разделение монолитного бэкенд-приложения (ЛР1) на микросервисы по документу, спроектированному в ДЗ4: три сервиса с отдельными базами данных, внутренние эндпоинты для их взаимодействия и gateway, через который публичный API остаётся доступен по тем же адресам, что и раньше.

Тема проекта: вариант 1 – приложение для бронирования столиков в ресторанах.

Ход работы

1. Исходные данные

Архитектура подробно спроектирована и обоснована в отчёте ДЗ4, поэтому здесь я только напоминаю основные решения, на которые опиралась при реализации.

Таблица 1 – Сервисы

Сервис	Порт	База данных	Отвечает за
auth	3001	auth_db	Пользователи, роли, выдача и проверка JWT
restaurants	3002	restaurants_db	Города, кухни, рестораны, фото, меню, столики, слоты, отзывы, администраторы ресторана
bookings	3003	bookings_db	Брони, поиск свободных столиков
gateway	8080	–	Маршрутизация публичного API

Каждый сервис подключается только к своей базе (database-per-service). Сервисы обращаются друг к другу по отдельному внутреннему HTTP API (7 эндпоинтов), запросы к которому защищены заголовком X-Internal-Token; эти адреса не публикуются через gateway и недоступны клиенту напрямую. Полное описание внутреннего API, схема архитектуры и обоснование границ сервисов – в отчёте ДЗ4 и в файле docs/openapi-internal.yaml.

2. Что нужно было реализовать в этой работе

По заданию курса в ЛР2 нужно было реализовать именно то, что было спроектировано в ДЗ4: разделить монолит на три сервиса с собственными базами данных, внутренние эндпоинты для их взаимодействия и gateway,

который делает разделение прозрачным для клиента – публичный API должен остаться прежним.

В эту работу не входило :

- Dockerfile и контейнеры для каждого сервиса – это задание ЛР3;
- очереди сообщений для связи сервисов – ДЗ5;
- развёртывание на удалённом сервере и HTTPS – ЛР4.

Из-за этого gateway я сделала как обычный Node-прокси внутри проекта, а не как отдельный контейнер nginx.

Критерий готовности, который я для себя взяла: все 43 операции публичного API из `docs/openapi.yaml` должны вести себя одинаково. Проверяла это прогоном тех же коллекций Postman, что использовались для ЛР1 (монолита).

3. Этапы написания кода

3.1 Каркас

Начала с каркаса: создала папки под каждый сервис и под gateway, `docker-compose.yml` с одним контейнером PostgreSQL, но с тремя базами внутри (`dbs/init.sql` создаёт три базы и трёх пользователей). Общие куски кода – JWT-мидлвар, обработчик ошибок – скопировала в каждый сервис отдельно: в проекте общий код не выносится в отдельный npm-пакет, а именно копируется, так что если что-то в этом коде поменяется, придется синхронизировать вручную во всех трёх местах.

3.2 auth-service

Auth сделала первым, потому что все остальные сервисы от него зависят – токены, пользователи, роли. Тут же завела внутренний API: сервисы общаются друг с другом напрямую по HTTP с отдельным заголовком `X-Internal-Token`, это не то же самое, что публичный API через gateway – эти адреса вообще наружу не торчат.

3.3 restaurants-service

Restaurants – самый большой кусок, 33 из 43 публичных операций: города, кухни, сами рестораны, столики, слоты, меню, отзывы, администраторы. Здесь пришлось повозиться с межсервисной логикой – например, когда администратор назначает кого-то администратором ресторана, нужно

обратиться в `auth` и повысить пользователю роль, а если запрашиваю список отзывов – тоже сходить в `auth` за именами авторов, потому что данные о пользователях в этой базе вообще не хранятся, только `id`.

3.4 *bookings-service*

Bookings – брони и поиск свободных столиков, здесь, наоборот, bookings ходит в restaurants за данными о столиках. И вот тут по ходу вылезла интересная особенность: раньше в монолите таблица бронирований напрямую ссылалась внешним ключом на таблицу столиков, а теперь это разные базы, и внешний ключ между сервисами сделать нельзя. Значит, перед тем как разрешить создать бронь, нужно реально сходить HTTP-запросом в другой сервис и спросить: «этот столик вообще существует и активен?».

3.5 *Связка и gateway*

Дальше был этап, который в плане ДЗ4 называется «связка», – доделала проверку, что нельзя удалить столик, если на нём висит бронь (снова сходить в bookings и спросить). А потом gateway – просто прокси, который смотрит на путь запроса и решает, к какому из трёх сервисов его перенаправить, по таблице маршрутов из ДЗ4.

4. Структура репозитория

Итоговая структура репозитория приведена ниже.

```
лр 2/
├─ docker-compose.yml      - контейнер Postgres (одна база-сервер, три БД)
├─ .env.example            - переменные контейнера Postgres (POSTGRES_*)
├─ dbs/
│   └─ init.sql            - создание трёх БД и пользователей при первом старте
│   └─ postgres-data/      - данные контейнера (не в git)
├─ docs/
│   └─ openapi.yaml        - публичный API, 43 операции
│   └─ openapi-internal.yaml - внутренний API, 7 операций
│   └─ postman-collections/ - коллекции и окружение Postman (ДЗ3)
├─ scripts/seed.js         - тестовые данные
├─ gateway/                - Node-прокси, порт 8080 (app.ts, routing.ts, proxy.ts)
├─ services/
│   └─ auth/               - порт 3001
│   └─ restaurants/        - порт 3002
│   └─ bookings/           - порт 3003
```

Каждый сервис в `services/*` устроен одинаково: `app.ts`, `config/`, `controllers/`, `dto/`, `models/`, `middlewares/`, `errors/`, `utils/`; у `restaurants` и `bookings` дополнительно есть `clients/` – HTTP-клиенты к другим сервисам, через которые и реализована связь, описанная в разделе 3.

5. Реакция сервисов на ошибки

Общее правило: если сервис вызывает другой сервис через внутренний API (заголовок `X-Internal-Token`) и не может до него достучаться, действуют три правила:

- нет соединения или ответ 5xx → клиенту возвращается 503;
- таймаут (3 секунды) → 504;
- GET-запрос при сбое повторяется один раз, POST и PATCH — нет.

Это касается большинства межсервисных вызовов: создание брони (bookings проверяет столик и слот в restaurants), поиск свободных столиков, назначение администратора ресторана (restaurants обращается в auth), список и просмотр брони, удаление столика, слота или ресторана (проверка через bookings). Если нужный сервис недоступен, вся операция падает с 503 или 504, ничего не выполняется наполовину.

Из этого правила есть два исключения (таблица 2), где вместо ошибки сервис отдаёт 200 с `null` в отдельных полях.

Таблица 2 – Исключения из общего правила

Где	Кто кого вызывает	Что происходит при сбое
Отзывы (список отзывов ресторана)	restaurants → auth	у каждого отзыва <code>user: null</code> вместо данных автора
История своих броней (GET /users/me/bookings)	bookings → restaurants	у каждой брони <code>restaurant</code> , <code>restaurant_table</code> , <code>time_slot</code> равны <code>null</code>

Логика в том, что в этих двух местах недостающие данные — это дополнение к уже имеющимся собственным данным: у сервиса и так есть сама бронь или сам отзыв в своей базе, и без имени автора или названия ресторана ответ не теряет смысла. Там, где чужие данные нужны не для дополнения, а для сути операции — проверить, что столик существует, что бронь принадлежит этому ресторану, что пользователь действительно есть, — деградации нет, только явная ошибка.

Это же правило распространяется и на gateway: если он проксирует запрос, а нужный сервис не отвечает, клиент тоже получает 503 или 504, а не зависание. Это поведение я проверила вручную, результаты — в разделе 6.2.

6. Проверка

6.1 Коллекции Postman из ДЗЗ

Обе коллекции Postman из ДЗЗ («1. Сценарии» и «2. По группам оренаpi») я прогнала через gateway, не меняя сами файлы коллекций – только окружение (baseUrl переключён на `http://localhost:8080/api/v1`).

Результат: «1. Сценарии» – 47 запросов, 70 проверок, 0 неудачных; «2. По группам оренаpi» – 50 запросов, 53 проверки, 0 неудачных.

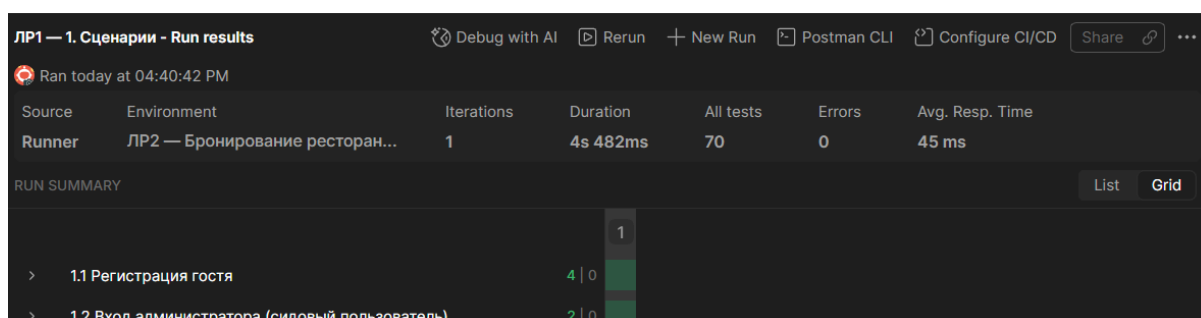


Рисунок 1 – Прогон коллекции «1. Сценарии» через gateway

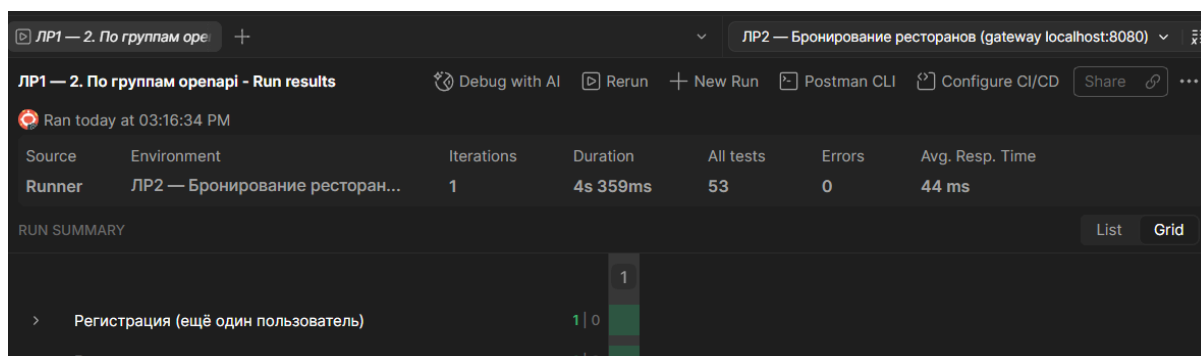


Рисунок 2 – Прогон коллекции «2. По группам оренаpi» через gateway

6.2 Проверка границ сервисов

Отдельно проверила, что разделение на сервисы не протекает наружу и что сервисы ведут себя при сбоях так, как описано в разделе 5.

Внутренний API снаружи недоступен:

```
curl.exe -i http://localhost:8080/internal/users/1
# ожидание: 404 – gateway не публикует /internal/*

curl.exe -i http://localhost:3001/internal/users/1
# ожидание: 401 – прямой запрос к auth без X-Internal-Token
```

Остановила restaurants-service и проверила, как ведут себя запросы, которым нужны его данные:

```
curl.exe -i http://localhost:8080/api/v1/restaurants
# ожидание: 503 – restaurants недоступен, общее правило

# GET /users/me/bookings – исключение: не 503, а null в отдельных полях
Invoke-RestMethod http://localhost:8080/api/v1/users/me/bookings `
-headers @{ Authorization = "Bearer $token" }
# ожидание: 200, но restaurant / restaurant_table / time_slot у каждой брони = null
```

Оба результата подтвердились: большинство запросов, которым нужны данные восстановленного сервиса, отвечают 503, и только два места (список отзывов ресторана и история броней пользователя) осознанно сделаны так, чтобы при сбое отдавать частичный ответ вместо ошибки – это отмечено как отступление от docs/openapi.yaml в README проекта.

```
PS C:\Users\User\Desktop\лабы по бэку\лр 2> curl.exe -i http://localhost:8080/internal/users/1
HTTP/1.1 404 Not Found
X-Powered-By: Express
Access-Control-Allow-Origin: *
Content-Type: application/json; charset=utf-8
Content-Length: 40
ETag: W/"28-6Dm+xM5lBUClAx583GPYtfsgOpE"
Date: Tue, 22 Sep 2026 13:46:55 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"statusCode":404,"message":"Not found"}
PS C:\Users\User\Desktop\лабы по бэку\лр 2> curl.exe -i http://localhost:3001/internal/users/1
HTTP/1.1 401 Unauthorized
X-Powered-By: Express
Access-Control-Allow-Origin: *
Content-Type: application/json; charset=utf-8
Content-Length: 77
ETag: W/"4d-+KlF+6ffZ4qa2XlNcA5A77tq60k"
Date: Tue, 22 Sep 2026 13:47:28 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"statusCode":401,"message":"Неверный внутренний ключ"}
```

```
PS C:\Users\User\Desktop\лабы по бэку\лр 2> curl.exe -i http://localhost:8080/api/v1/restaurants
HTTP/1.1 503 Service Unavailable
X-Powered-By: Express
Access-Control-Allow-Origin: *
Content-Type: application/json; charset=utf-8
Content-Length: 50
ETag: W/"32-+v+3jPldDYzsc4flo+Ni39muFJw"
Date: Tue, 22 Sep 2026 13:49:37 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"statusCode":503,"message":"Service unavailable"}
```

Рисунок 3 – Внутренний API недоступен снаружи, сбой сервиса даёт 503

6.3 Независимость баз

Подключилась к контейнеру через psql и посмотрела список баз:

```
docker exec -it 2-db-1 psql -U postgres -l
```

В списке – три отдельные базы: `auth_db`, `restaurants_db`, `bookings_db`, у каждой свой пользователь (`auth_user`, `restaurants_user`, `bookings_user` из `dbs/init.sql`). Затем проверила таблицу `booking`:

```
docker exec -it 2-db-1 psql -U bookings_user -d bookings_db
\d booking
```

В `booking` нет внешних ключей на `restaurant_table` и `time_slot` – раньше в монолите они были, а теперь это обычные целочисленные поля; вместо них есть новое поле `restaurant_id`. Так на уровне схемы видно, что прямой связи между базами больше нет – всё, что раньше проверялось внешним ключом, теперь проверяется HTTP-запросом к другому сервису (раздел 3.4).

7. Инструкции к запуску

Пошаговая инструкция по установке зависимостей, настройке `.env`, запуску всех сервисов и базы данных, остановке и четырём сценариям повторного запуска (с теми же данными или с чистой базой), список тестовых пользователей и частых проблем – в файле `README.md` в корне репозитория (лр 2/`README.md`). Здесь не дублирую её содержимое, только напоминаю общий порядок: `docker compose up -d` → четыре сервиса (`auth`, `restaurants`, `bookings`, `gateway`) в отдельных окнах терминала → `node scripts/seed.js`.

Вывод

В ходе работы монолит из ЛР1 разделён на три микросервиса (auth, restaurants, bookings) с отдельными базами данных по плану, спроектированному в Д34, и общий gateway делает это разделение прозрачным для клиента: публичный API не изменился, что подтверждено прогоном коллекций Postman из Д33 – они прошли через gateway с тем же результатом, что и на монолите. При реализации пришлось заменить внешние ключи между сущностями, которые оказались в разных базах, на HTTP-запросы к другим сервисам (проверка столика и слота при бронировании, проверка брони перед удалением, получение данных пользователя), а также решить, как сервисы ведут себя при недоступности друг друга. Дальше эти же сервисы будут связаны очередью сообщений в Д35.